

Valdir de Souza Carvalho Neto

ENP 03 - Estrutura de um Sistema Operacional

Sistemas Operacionais

São João Evangelista - MG

2026

Sumário

1	RESOLUÇÃO DOS EXERCÍCIOS	2
1.1	O que diferencia o <i>núcleo</i> do restante do sistema operacional? . . .	2
1.2	Seria possível construir um sistema operacional seguro usando um processador que não tenha níveis de privilégio? Por quê?	2
1.3	Os processadores da família x86 possuem dois bits para definir o nível de privilégio, resultando em 4 níveis distintos. A maioria dos sistemas operacionais para esses processadores usam somente os níveis extremos (0 e 3, ou 00 e 11). Haveria alguma utilidade para os níveis intermediários?	3
1.4	Quais as diferenças entre <i>interrupções</i> , <i>exceções</i> e <i>traps</i> ?	3
1.5	O comando em linguagem C <code>fopen</code> é uma chamada de sistema ou uma função de biblioteca? Por quê?	4
1.6	A operação em modo usuário permite ao processador executar somente parte das instruções disponíveis em seu conjunto de instruções. Quais das seguintes operações não deveriam ser permitidas em nível usuário? Por quê?	4
1.7	Considerando um processo em um sistema operacional com proteção de memória entre o núcleo e as aplicações, indique quais das seguintes ações do processo teriam de ser realizadas através de chamadas de sistema, justificando suas respostas:	5
1.8	Coloque na ordem correta as ações abaixo, que ocorrem durante a execução da função <code>printf("Hello world")</code> por um processo (observe que nem todas as ações indicadas fazem parte da sequência).	6
	REFERÊNCIAS	8

1 Resolução dos Exercícios

1.1 O que diferencia o *núcleo* do restante do sistema operacional?

O núcleo (ou *kernel*) é o coração do sistema operacional, responsável pela gerência dos recursos do hardware usados pelas aplicações. Ele também implementa as principais abstrações utilizadas pelos aplicativos e programas utilitários. O núcleo executa em um modo especial de operação do processador, denominado modo privilegiado ou modo sistema, tendo pleno acesso ao hardware, podendo configurá-lo e gerenciá-lo.

Os demais componentes do sistema operacional (drivers, programas utilitários, código de inicialização) possuem funções distintas. Os drivers são módulos de código específicos para acessar os dispositivos físicos. Os programas utilitários facilitam o uso do sistema computacional, fornecendo funcionalidades complementares ao núcleo, como formatação de discos, manipulação de arquivos, interpretador de comandos, interface gráfica, etc. O código de inicialização (*boot code*) realiza tarefas como reconhecer os dispositivos instalados, testá-los, configurá-los e carregar o núcleo em memória. Enquanto o núcleo e os drivers executam em modo privilegiado, os utilitários e aplicações executam em modo usuário, com acesso mais restrito ao hardware.

1.2 Seria possível construir um sistema operacional seguro usando um processador que não tenha níveis de privilégio? Por quê?

Não seria possível. Os níveis de privilégio permitem diferenciar os privilégios de execução dos diferentes tipos de software. Enquanto o núcleo e os drivers devem ter pleno acesso ao hardware para poder configurá-lo e gerenciá-lo, os aplicativos e utilitários devem ter acesso mais restrito a ele, para não interferir nas configurações e na gerência, o que poderia desestabilizar o sistema inteiro. Além disso, aplicações com acesso pleno ao hardware seriam um risco à segurança, pois poderiam contornar facilmente os mecanismos de controle de acesso aos recursos (tais como arquivos e áreas de memória).

Sem os níveis de privilégio, não haveria como impor essa diferenciação. Os *flags* que definem o nível de privilégio só podem ser modificados por código executando no nível núcleo, o que impede usuários maliciosos de contornar essa barreira de proteção. Portanto, os níveis de privilégio são essenciais para garantir a estabilidade e a segurança do sistema.

1.3 Os processadores da família x86 possuem dois bits para definir o nível de privilégio, resultando em 4 níveis distintos. A maioria dos sistemas operacionais para esses processadores usam somente os níveis extremos (0 e 3, ou 00 e 11). Haveria alguma utilidade para os níveis intermediários?

Os processadores no padrão Intel x86 dispõem de 4 níveis de privilégio (0 a 3, sendo 0 o nível mais privilegiado). Entretanto, a maioria dos sistemas operacionais construídos para esses processadores só usa os níveis extremos (0 para o núcleo do sistema operacional e 3 para utilitários e aplicações).

O texto estabelece que, na forma mais simples desse esquema, podemos considerar dois níveis básicos de privilégio: nível núcleo (também denominado nível supervisor, sistema, monitor ou ainda *kernel space*) e nível usuário (também chamado *userspace*). Embora o material indique que os níveis intermediários (1 e 2) não são usados pela maioria dos sistemas operacionais, a existência de 4 níveis sugere que seria possível uma estrutura com maior granularidade de privilégios, onde os níveis intermediários poderiam ser utilizados para drivers ou serviços do sistema com privilégios parciais, embora esta aplicação específica não seja detalhada no capítulo.

1.4 Quais as diferenças entre *interrupções*, *exceções* e *traps*?

- **Interrupções:** São eventos gerados por dispositivos periféricos. Cada controlador de dispositivo pode notificar o processador enviando uma requisição de interrupção (IRQ – *Interrupt ReQuest*) através do barramento de controle. A maioria das interrupções recebidas pelo processador têm como origem eventos externos a ele, ocorridos nos dispositivos periféricos e reportados por seus controladores. Ao receber a requisição, o processador suspende seu fluxo de execução corrente e desvia para uma rotina de tratamento de interrupção (*interrupt handler*). Para distinguir interrupções geradas por dispositivos distintos, cada interrupção é identificada pelo hardware por um número inteiro.
- **Exceções:** São eventos gerados pelo próprio processador usando o mesmo mecanismo das interrupções. Ações como instruções ilegais (inexistentes ou com operandos inválidos), tentativas de divisão por zero ou outros erros de software disparam exceções no processador, que resultam na ativação de uma rotina de tratamento de exceção, usando o mesmo mecanismo das interrupções (e a mesma tabela de endereços de funções).

- **Traps (interrupções de software):** São um mecanismo específico implementado por uma instrução do processador que usa o mesmo mecanismo das interrupções para ativar rotinas do núcleo. Ao ser executada, essa instrução comuta o processador para o nível núcleo e executa o código contido em uma rotina predefinida, de forma similar a uma interrupção de hardware. Por essa razão, esse mecanismo é denominado interrupção de software, ou *trap*. A ativação de uma rotina do núcleo usando esse mecanismo é denominada chamada de sistema (*system call* ou *syscall*).

1.5 O comando em linguagem C `fopen` é uma chamada de sistema ou uma função de biblioteca? Por quê?

A função `fopen` é uma **função de biblioteca**. Geralmente as chamadas de sistema são oferecidas para as aplicações em modo usuário através de uma biblioteca do sistema (*system library*), que prepara os parâmetros, invoca a chamada e, no retorno desta, devolve à aplicação os resultados obtidos. O texto menciona que o conjunto de chamadas de sistema inclui operações de gestão de arquivos: criar, remover, abrir, fechar, ler, escrever, ler/mudar atributos.

A função `fopen` da biblioteca padrão C é uma interface de mais alto nível que, internamente, utiliza as chamadas de sistema de gestão de arquivos (como `open`) para realizar a abertura do arquivo. A `fopen` prepara os parâmetros adequadamente e invoca a chamada de sistema correspondente, atuando como intermediária entre a aplicação e o núcleo.

1.6 A operação em modo usuário permite ao processador executar somente parte das instruções disponíveis em seu conjunto de instruções. Quais das seguintes operações não deveriam ser permitidas em nível usuário? Por quê?

No nível usuário (*userspace*), somente um subconjunto das instruções do processador e registradores estão disponíveis. Instruções consideradas “perigosas”, como RESET (reiniciar o processador) e IN/OUT (acessar portas de entrada/saída), são proibidas. Caso o código em execução tente executar uma instrução proibida, o hardware irá gerar uma exceção, desviando a execução para uma rotina de tratamento dentro do núcleo.

Com base nisso, as operações que não deveriam ser permitidas em nível usuário são:

- a) **Ler uma porta de entrada/saída** – Não permitida, pois o texto indica que instruções de acesso a portas de entrada/saída (como IN e OUT) são proibidas em modo usuário.
- b) **Efetuar uma divisão inteira** – Permitida, é uma operação aritmética normal.
- c) **Escrever um valor em uma posição de memória** – Permitida, desde que dentro da área de memória do próprio processo.
- d) **Ajustar o valor do relógio do hardware** – Não permitida, pois envolve acesso a portas de entrada/saída (instruções IN/OUT) para acessar o hardware.
- e) **Ler o valor dos registradores do processador** – Não permitida, pois o texto estabelece que no nível usuário somente um subconjunto dos registradores está disponível. Registradores de controle e configuração do hardware são inacessíveis.
- f) **Mascarar uma ou mais interrupções** – Não permitida, pois trata-se de uma operação de controle do hardware que afeta o funcionamento do sistema como um todo.

Justificativa geral: Enquanto o núcleo e os drivers devem ter pleno acesso ao hardware para poder configurá-lo e gerenciá-lo, os aplicativos e utilitários devem ter acesso mais restrito a ele, para não interferir nas configurações e na gerência, o que poderia desestabilizar o sistema inteiro. Ademais, aplicações com acesso pleno ao hardware seriam um risco à segurança, pois poderiam contornar facilmente os mecanismos de controle de acesso aos recursos.

1.7 Considerando um processo em um sistema operacional com proteção de memória entre o núcleo e as aplicações, indique quais das seguintes ações do processo teriam de ser realizadas através de chamadas de sistema, justificando suas respostas:

- a) **Ler o relógio de tempo real do hardware.** (*Chamada de sistema*) – Acessar o hardware do relógio é uma operação de entrada/saída que envolve portas de hardware. Instruções de acesso a portas de entrada/saída são proibidas em modo usuário. Ademais, a ação de “ler/mudar data e hora” faz parte da gestão do sistema, realizada via chamadas de sistema.
- b) **Enviar um pacote através da rede.** (*Chamada de sistema*) – Enviar um pacote exige acessar o controlador de rede (placa Ethernet) através de portas de

entrada/saída. O acesso aos dispositivos se dá através das portas de entrada/saída, e este acesso é restrito ao modo núcleo. A ação de “receber/enviar dados” é parte das chamadas de sistema de comunicação.

- c) **Calcular uma exponenciação.** (*Não necessita*) – É uma operação puramente computacional realizada pela ULA (unidade lógica e aritmética) do processador, sem necessidade de acesso ao hardware ou a recursos gerenciados pelo núcleo.
- d) **Preencher uma área de memória do processo com zeros.** (*Não necessita*) – O núcleo configura a MMU para criar uma área de memória exclusiva para cada aplicação. Dentro de sua própria área de memória, o processo pode ler e escrever livremente sem necessidade de chamadas de sistema.
- e) **Remover um arquivo do disco.** (*Chamada de sistema*) – As operações de “criar, remover, abrir, fechar, ler, escrever” arquivos fazem parte da gestão de arquivos realizadas via chamadas de sistema. Remover um arquivo exige interagir com o sistema de arquivos e o controlador de disco, recursos gerenciados exclusivamente pelo núcleo.

1.8 Coloque na ordem correta as ações abaixo, que ocorrem durante a execução da função `printf("Hello world")` por um processo (observe que nem todas as ações indicadas fazem parte da sequência).

A sequência correta, baseada no roteiro típico de uma chamada de sistema e adaptada para o `printf` que utiliza a chamada de sistema `write`, é:

1. **(h)** O processo chama a função `printf` da biblioteca C.
2. **(c)** A função de biblioteca `printf` recebe e processa os parâmetros de entrada (a string “Hello world”).
3. **(d)** A função de biblioteca `printf` prepara os registradores para solicitar a chamada de sistema `write()`.
4. **(g)** Uma interrupção de software é acionada.
5. **(a)** A rotina de tratamento da interrupção de software é ativada dentro do núcleo.
6. **(i)** A operação de escrita no terminal é efetuada ou agendada pela rotina de tratamento da interrupção.

7. (b) A função `printf` finaliza sua execução e devolve o controle ao código do processo.

Justificativa da sequência: De acordo com o roteiro descrito no material: (1) a aplicação invoca a função da biblioteca; (2) a função preenche os registradores com os parâmetros e o *opcode*; (3) a função invoca a chamada de sistema através da instrução `syscall`, que é a interrupção de software; (4) o processador comuta para o nível privilegiado e transfere o controle para a rotina de entrada do núcleo; (5-6) a função dentro do núcleo efetua a operação desejada; (7-8) ao final, os valores de retorno são escritos e o processamento retorna à função em modo usuário, que finaliza e devolve o controle à aplicação.

Ações que não fazem parte da sequência:

- (e) O disco rígido gera uma interrupção indicando a conclusão da operação – Não faz parte porque a escrita no terminal não envolve diretamente o disco rígido.
- (f) O escalonador escolhe o processo mais prioritário para execução – Não faz parte imediata desta sequência. Isto só ocorreria se a operação não pudesse ser concluída imediatamente (como na leitura de teclado).
- (j) O controle volta para a função `printf` em modo usuário – É um passo implícito que ocorre entre (i) e (b).

Referências

MAZIERO, C. A. *Sistemas Operacionais: Conceitos e Mecanismos*. Edição do autor, 2019. ISBN 978-85-7335-340-2. Disponível em: <<https://wiki.inf.ufpr.br/maziero/lib/exe/fetch.php?media=socm:socm-livro.pdf>>. Nenhuma citação no texto.